Big Data y Minería de Datos

Unidad 2. Reglas en la minería de datos

U2.1. Introducción a los fundamentos lógicos

UNIVERSIDAD DE
MURCIA

Unidad 2. Reglas en la minería de datos
Estructura

UNIVERSIDAD DE MURCIA

- Representación del conocimiento
- Introducción a la lógica
- Lógica Proposicional (LP)
- Métodos de inferencia en LP
- Lógica de Primer Orden (LPO)
- Programación Declarativa

Big Data y Minería de Datos
Contenidos

UNIVERSIDAD DE MURCIA

Introducción a la minería de datos

Técnicas de inteligencia artificial

Reglas

Fundamentos lógicos

Sistemas basados en reglas

Reglas de asociación

Aprendizaje computacional

Fundamentos y evaluación

Regresión y clasificación

No supervisado

Procesado del lenguaje natural

Fundamentos

Representación y tópicos

Sentimientos y NER

Introducción al big data

UNIVERSIDAD DE MURCIA

Unidad 2. Reglas en la minería de datos

REPRESENTACIÓN DEL CONOCIMIENTO

Unidad 2. Reglas en la minería de datos
Definiciones básicas

UNIVERSIDAD DE MURCIA

- Def. Dominio: es el entorno o contexto de trabajo o se quiere desarrollar un sistema basado en IA.
- Def. Conocimiento: Desde un punto vista de la IA y la ingeniería, es la comprensión teórica y práctica de un tema o un dominio.
- Def. Expertos: aquellos que poseen un conocimiento profundo sobre un dominio específico.

Tipos de conocimiento

- Hay muchas taxonomías para clasificar el conocimiento.
- Conocimiento Explícito vs. Tácito
  - **Explícito**: aquel conocimiento que se puede encontrar de forma evidente y estructurada (en un libro, manual, etc.)
  - **Tácito**: conocimiento difícil de expresar y que implica diferentes destrezas y no se encuentra de forma sencilla y explícita (ej. montar en bicicleta, diagnosticar una enfermedad)

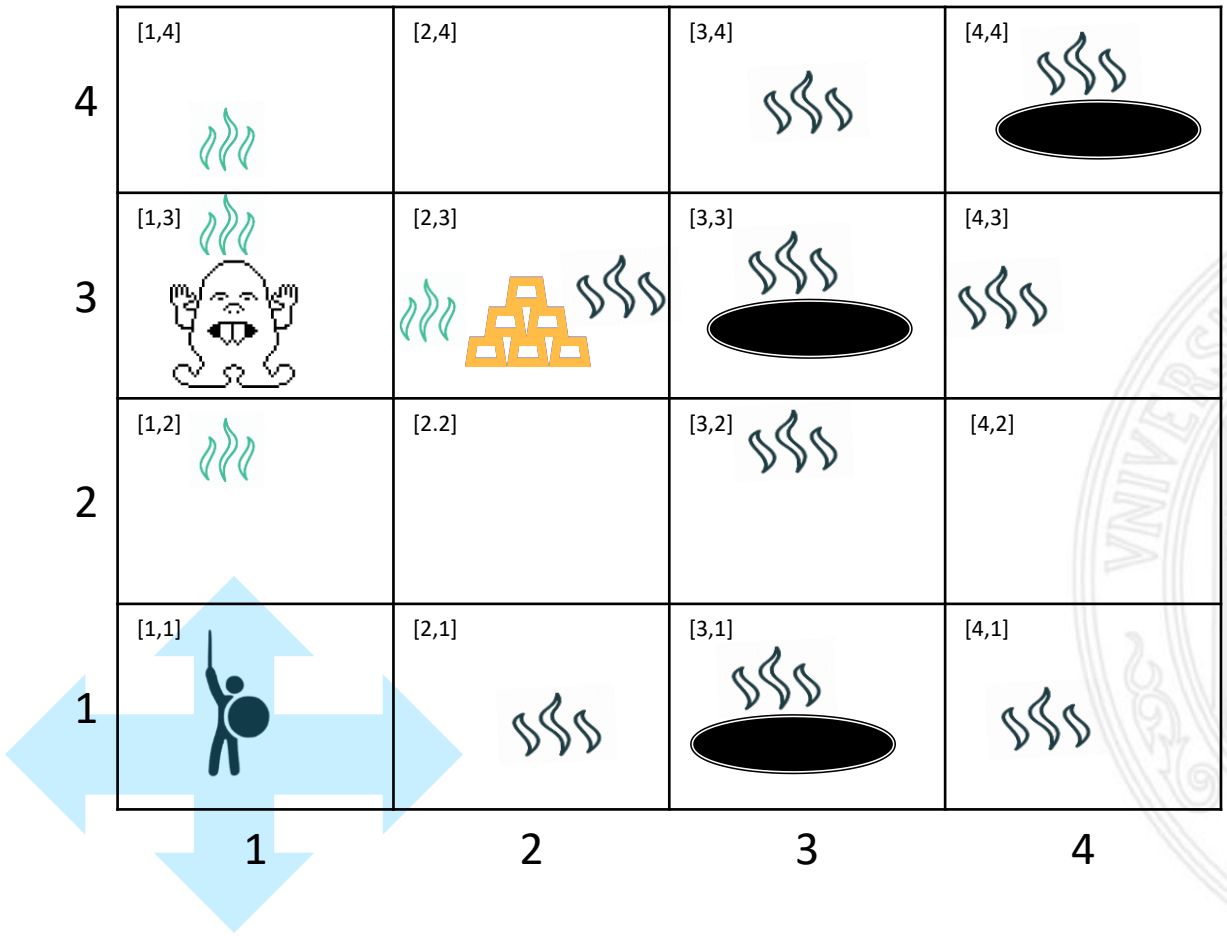
- Los sistemas de IA necesitan resolver problemas complejos y requieren conocimiento
- Para su representación hay que:
  - **Conceptualización**: representar todas las características, relaciones e interacciones del problema
  - **Razonamiento**: la representación debe poder ser manipulado para resolver problemas
- La extracción del conocimiento es el cuello de botella
  - **Fuentes de expertos**: requiere de técnicas de entrevistas específicas, adquisición de información, validación, verificación
  - **Fuentes de datos**: técnicas de aprendizaje para extraer datos de conocimiento inherente (patrones ocultos)

UNIVERSIDAD DE MURCIA

Unidad 2. Reglas en la minería de datos

INTRODUCCIÓN A LA LÓGICA


Wumpus
Hedor
Agujero
Brisa
Oro
Heroe
Seguro
Visited


[1,4]
[2,4]
[3,4]
[4,4]
[1,3]
[2,3]
[3,3]
[4,3]
[1,2]
[2.2]
[3,2]
[4,2]
[1,1]
[2,1]
[3,1]
[4,1]
4
3
2
1
1
2
3
4

Unidad 2. Reglas en la minería de datos
¿Qué es la lógica?

- El estudio de la verdad de una sentencia en base a un modelo.
- Tiene los siguientes componentes:
  - Elementos léxicos:
    - Que será símbolos validos
    - Una sentencia debe estar compuesta por símbolos válidos
  - Sintaxis:
    - La gramática: reglas para construir una sentencia bien formada
    - Sin significado, completamente una estructura
  - Semántica:
    - Significado de los símbolos/sentencias en un mundo potencial (Modelo)
    - Valores asociados a una sentencia = verdad o falso

- Podríamos tener unos componentes como puede ser x+y=6
- Elementos léxicos:
  - Símbolos = {x,y,+, -,*,/,=,1,2,3,4,5,6,7,8,9,0}
- Sintaxis
  - “x6+z=“ no está bien formada
  - “x+z=6” está bien formada
- Semántica
  - “x+z=6” es verdad en un mundo donde x=3 y z=3 (modelo)
  - “x+z=6” es falso en un mundo donde x=2 y z=3 (otro modelo)
- Si una sentencia α es verdad en un modelo *m*, entonces decimos que *m* satisface α

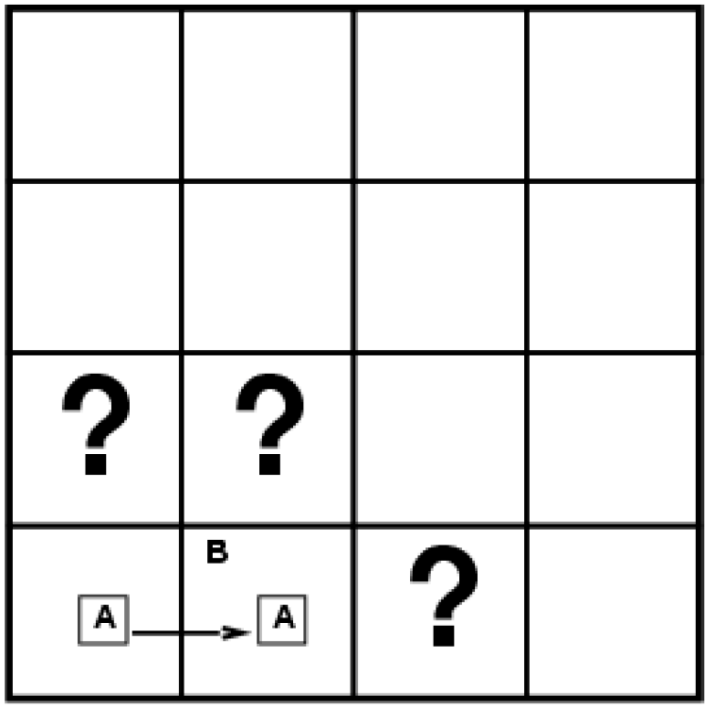
- Una sentencia es satisfacible si existe al menos un modelo que hace que la sentencia sea cierta
- Tipos de sentencias
  - Tautología (válida): la sentencia es siempre verdad (para todos los modelos
  - Contradicción: la sentencia es siempre falsa (para todos los modelos)
  - Contingente: la sentencia es verdad o falsa dependiendo del modelo

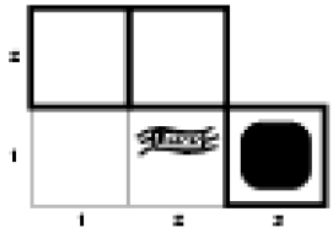
Unidad 2. Reglas en la minería de datos
Implicación (consecuencia o consistencia)

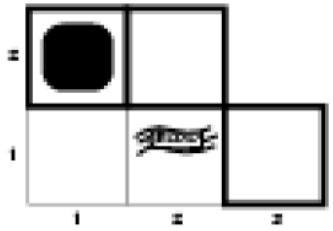
UNIVERSIDAD DE MURCIA

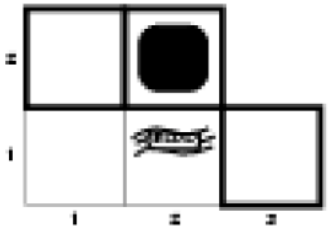
- El corazón del razonamiento lógico
- Si una nueva sentencia β se puede derivar de una existente α (premisa), entonces β es una consecuencia lógica de α (α⊨β)
  - Esto es: $\alpha \models \beta \Leftrightarrow M(\alpha) \subseteq M(\beta)$
  - Por ej,. x=0 implica x*y=0
  - Por ej, una KB contiene “Mariano no ganó” y “Mariano ganó o José ganó” implica…
- Relación entre sentencias (sintaxis) basada en la semántica
- Inconsistencia: Hipótesis cierta lleva a un consecuente falso
- Equivalencia: Dos sentencias que cada una de ellas es implicada de la otra: $\alpha \models \beta \wedge \beta \models \alpha \Leftrightarrow \alpha \equiv \beta$

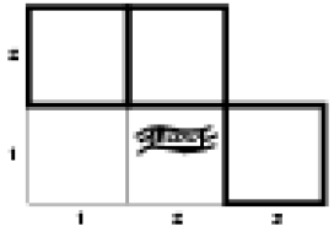
- Situación después de no detectar nada en [1,1] y moverse a la derecha, con brisa en [2,1]. Considerar solo agujeros

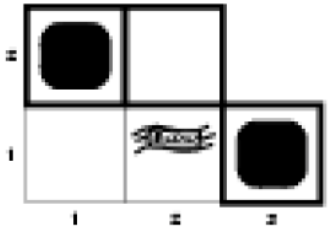


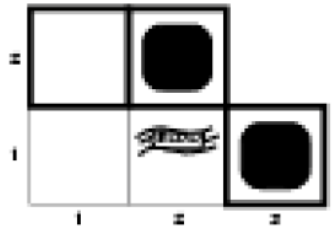


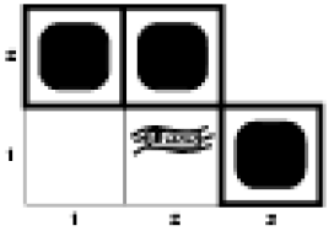


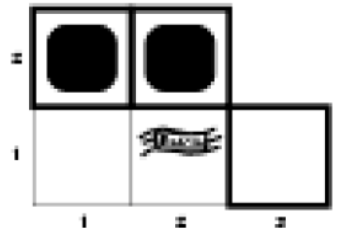












Inferencia (razonamiento o cálculo)

- El proceso de derivar nuevas sentencias de las antiguas dado un conocimiento cierto: $KB \vdash_i \alpha$
- Dada nuestra base de conocimiento KB (verdad), se deriva α usando el algoritmo/mecanismo *i*
- Mientras que la implicación indica que se puede derivar, la inferencia explica el cómo hacerlo (ejemplo del pajar)
- Propiedad corrección (*truth preserving o sound*)
  - Un algoritmo (*i*) es correcto, solo si las sentencias que deriva son consecuencias lógicas de la KB, *i* preserva la verdad
- Completitud
  - Un algoritmo (*i*) es completo, si es capaz de derivar cualquier sentencia lógicamente válida a partir de la KB
  - Metáfora: *i* hace análisis sistemático hasta encontrar la aguja

UNIVERSIDAD DE MURCIA

Unidad 2. Reglas en la minería de datos

LÓGICA PROPOSICIONAL

Lógica Proposicional: Sintaxis

- Es la lógica más simple para ilustrar ideas básicas
- Los símbolos de las proposiciones (P_1, P_2..) son sentencias
- Pueden ser verdaderas o falsas
- Si S es una sentencia, ¬S es una sentencia (negación)
- Si S_1 y S_2 son sentencias:
  - S_1 ^ S_2 es una sentencia (conjunción)
  - S_1 v S_2 es una sentencia (disyunción)
  - $S_1 \Rightarrow S_2$ es una sentencia (implicación)
  - $S_1 \Leftrightarrow S_2$ es una sentencia (bicondicional)



- Cada modelo especifica true/false para cada símbolo proposicional
  - Reglas para evaluar la verdad con respecto de un modelo *m*

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	*false*	*true*	*false*	*false*	*true*	*true*
false	*true*	*true*	*false*	*true*	*true*	*false*
true	*false*	*false*	*false*	*true*	*false*	*false*
true	*true*	*false*	*true*	*true*	*true*	*true*

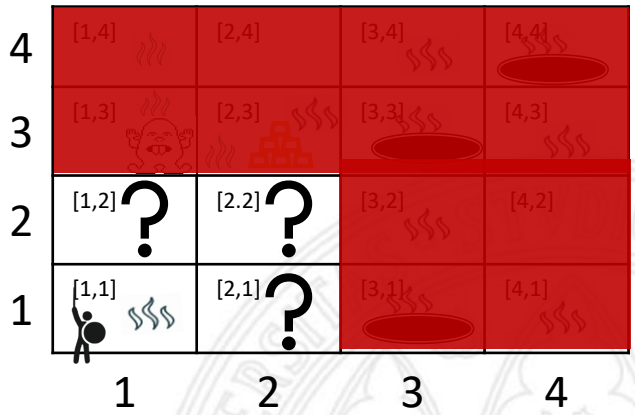
- Por ejemplo (3 proposiciones): $P_{1,2}$ (false), $P_{2,2}$ (true), $P_{3,1}$ (false)
- Con esto símbolos se pueden obtener 8 ($2^{\wedge 3}$) posibles modelos
- Se pueden aplicar procesos recursivos para evaluar una sentencia arbitraria:
  - $\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1}) = true \wedge (true \vee false) = true \wedge true = true$

- Ejercicio 1. Crear tabla de la verdad para las proposiciones $P_{1,2}$, $P_{1,2}$, $P_{3,1}$; y resuelve ¬ $P_{1,2}$ ^ ($P_{2,2}$ v $P_{3,1}$) :

$P_{1,2}$	$P_{2,2}$	$P_{3,1}$	$\neg P_{1,2}$	$P_{2,2} \vee P_{3,1}$	$\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1})$

UNIVERSIDAD DE MURCIA

- *Semántica:*
  - $P_{i,j}$ (Pit) agujero en (i,j)
  - $B_{i,j}$ (Breeze) brisa en (i,j)
  - $S_{i,j}$ (Stench) hedor en (i,j)
  - $W_{i,j}$ (Wumpus) en (i,j)
  - R_i i-estima sentencia



- Lo que sabemos:
  - Hay brisa en (1,1) $KB = \{B_{1,1}\}$
  - La brisa está producida por agujero cercano

$B_{1,1} \Rightarrow P_{1,2} \vee P_{2,1}$?

$B_{1,1} \Leftrightarrow P_{1,2} \vee P_{2,1}$

Unidad 2. Reglas en la minería de datos
Lógica Proposicional: Semántica en Wumpus

UNIVERSIDAD DE MURCIA



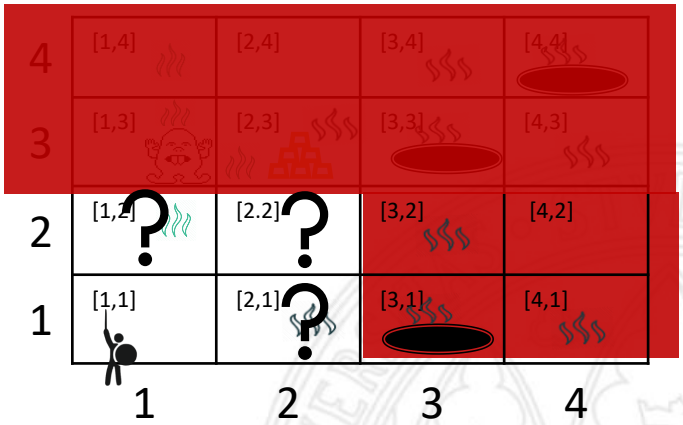
$P_{i,j}$ Pit in (i,j)

$B_{i,j}$ Breeze in (i,j)

$S_{i,j}$ Stench in (i,j)

$W_{i,j}$ Wumpus in (i,j)

R_i i-th sentence



- $R_1: \neg P_{1,1}$
- $R_2: B_{1,1} \Leftrightarrow P_{1,2} \vee P_{2,1}$
- $R_3: B_{2,1} \Leftrightarrow P_{1,1} \vee P_{2,2} \vee P_{3,1}$
- $R_4: \neg B_{1,1}$

- $R_5: \neg P_{1,2}$
- $R_6: \neg P_{2,1}$

DECISIÓN: MOVER A DERECHA CELDA [2,1]

Unidad 2. Reglas en la minería de datos
Lógica Proposicional: Semántica en Wumpus

UNIVERSIDAD DE MURCIA



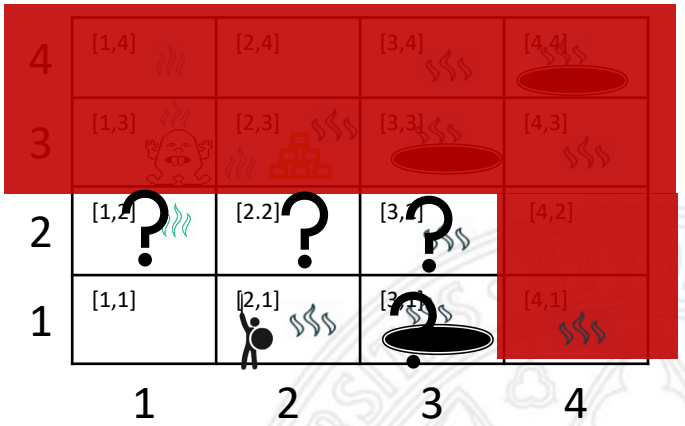
$P_{i,j}$ Pit in (i,j)

$B_{i,j}$ Breeze in (i,j)

$S_{i,j}$ Stench in (i,j)

$W_{i,j}$ Wumpus in (i,j)

R_i i-th sentence



- $R_1: \neg P_{1,1}$
- $R_2: B_{1,1} \Leftrightarrow P_{1,2} \vee P_{2,1}$
- $R_3: B_{2,1} \Leftrightarrow P_{1,1} \vee P_{2,2} \vee P_{3,1}$
- $R_4: \neg B_{1,1}$
- $R_5: \neg P_{1,2}$
- $R_6: \neg P_{2,1}$

- $R_7: B_{2,1}$
- $R_3: B_{2,1} \Leftrightarrow P_{1,1} \vee P_{2,2} \vee P_{3,1}$
- $R_1: \neg P_{1,1}$
- $R_8: P_{2,2} \vee P_{3,1}$

DECISION: VOLVER A [1,1] Y MOVER A LA CELDA [1,2]

Unidad 2. Reglas en la minería de datos
Lógica Proposicional: Semántica en Wumpus

UNIVERSIDAD DE MURCIA



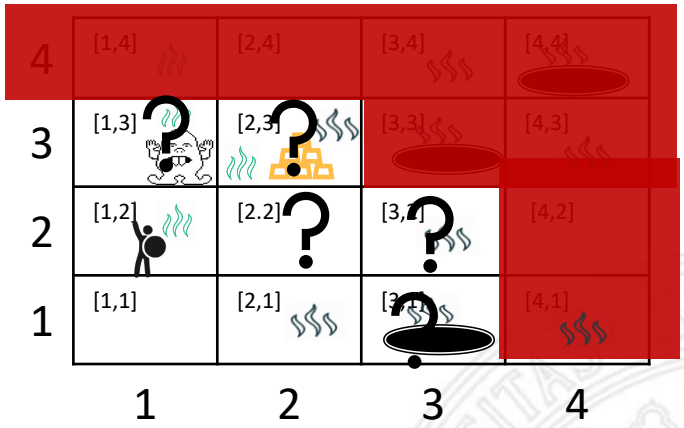
$P_{i,j}$ Pit in (i,j)

$B_{i,j}$ Breeze in (i,j)

$S_{i,j}$ Stench in (i,j)

$W_{i,j}$ Wumpus in (i,j)

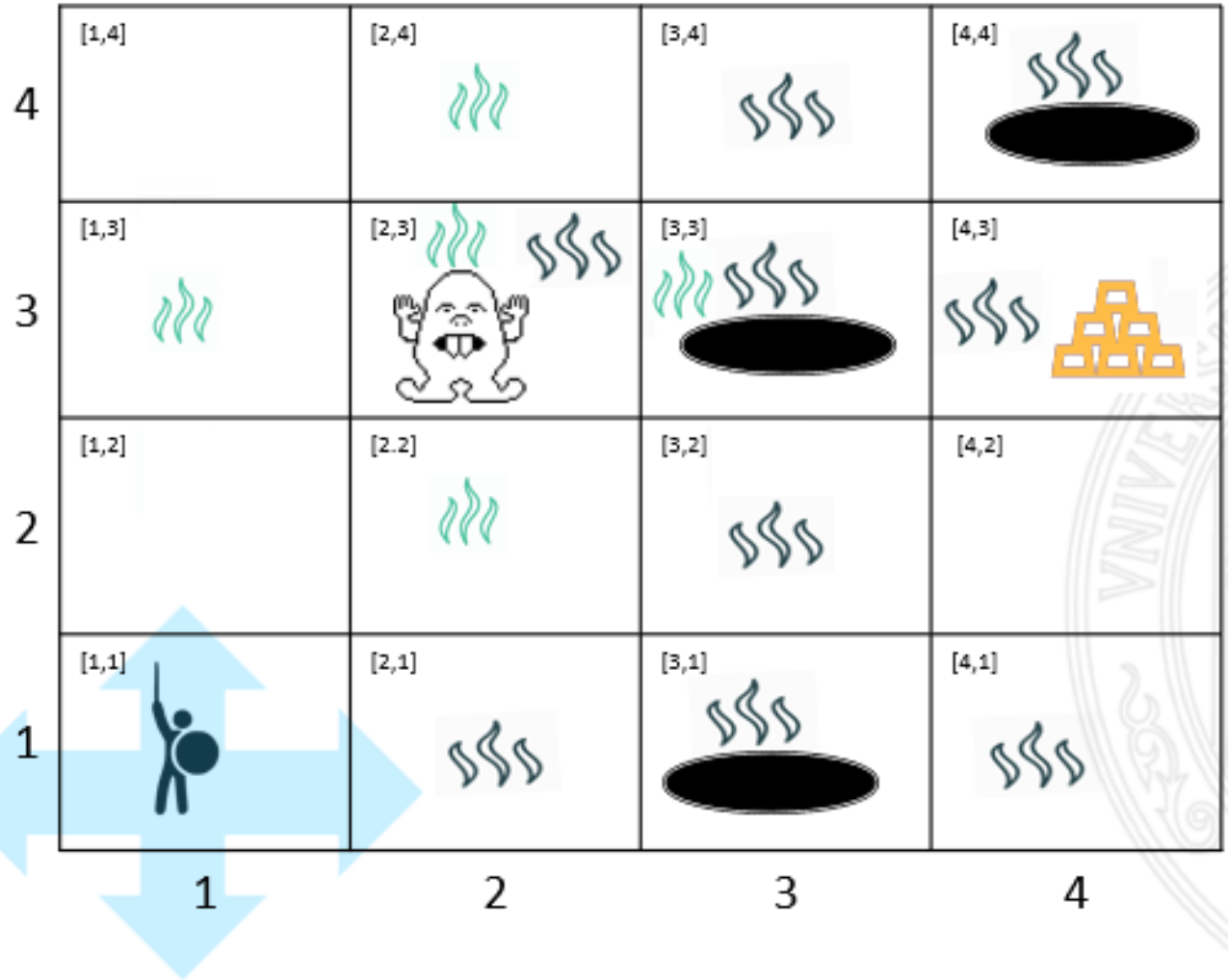
R_i i-th sentence



- $R_1: \neg P_{1,1}$
- $R_2: B_{1,1} \Leftrightarrow P_{1,2} \vee P_{2,1}$
- $R_3: B_{2,1} \Leftrightarrow P_{1,1} \vee P_{2,2} \vee P_{3,1}$
- $R_4: \neg B_{1,1}$
- $R_5: \neg P_{1,2}$
- $R_6: \neg P_{2,1}$
- $R_7: B_{2,1}$
- $R_8: P_{2,2} \vee P_{3,1}$
- $R_9: S_{1,2}$
- $R_{10}: S_{1,2} \Leftrightarrow W_{1,1} \vee W_{2,2} \vee W_{1,3}$
- $R_9: \neg W_{1,1}$
- $R_{10}: W_{2,2} \vee W_{1,3}$
- $R_{11}: S_{2,1} \Leftrightarrow W_{1,1} \vee W_{2,2} \vee W_{3,1}$
- $R_{12}: \neg S_{2,1}$
- $R_{13}: \neg W_{2,2}$
- $\boldsymbol{R_{14}: W_{1,3}}$

UNIVERSIDAD DE MURCIA

- Ejercicio 2. Resuelve este mapa.



$P_{i,j}$ Pit in (i,j)

$B_{i,j}$ Breeze in (i,j)

$S_{i,j}$ Stench in (i,j)

$W_{i,j}$ Wumpus in (i,j)

R_i i-th sentence

UNIVERSIDAD DE MURCIA



Unidad 2. Reglas en la minería de datos

MÉTODOS DE INFERENCIA EN LP

Unidad 2. Reglas en la minería de datos
Métodos de inferencia

UNIVERSIDAD DE MURCIA

- Tablas de verdad (model-checking)
- Deducción (demonstrador)
- Resolución (demostrador)
- **Encadenamiento (demostrador)**
- DPLL y WalkSAT (model-checking eficiente)
- Tableaux

- Algoritmo de encadenamiento hacia adelante
- El algoritmo determina si una proposición *q* (la cuestión a demonstrar) se puede inferida lógicamente a partir de la KB
- Comenzamos con un conjunto de literales positivos en la KB
  - Para todas las reglas conocidas (implicación), añadir la HEAD (conclusiones) a la KB, si el ODY (antecedentes) se satisfacen
  - El algoritmo acaba cuando *q* se añade a la KB o bien cuando no se pueden aplicar más inferencias (no hay más reglas que se puedan aplicar)

Unidad 2. Reglas en la minería de datos
Ejemplo encadenamiento hacia adelante

UNIVERSIDAD DE MURCIA

Base de conocimiento (KB) - Reglas

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

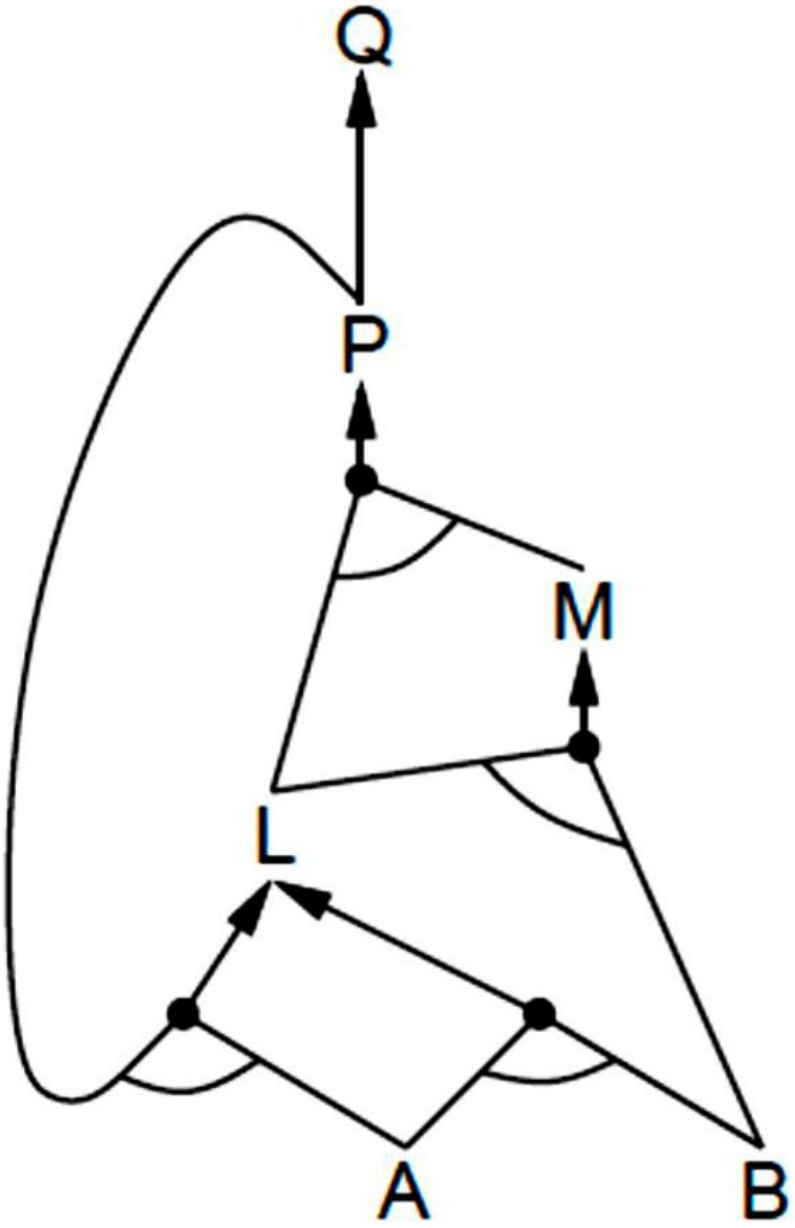
$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

Base de conocimiento (KB) - Hechos Iniciales

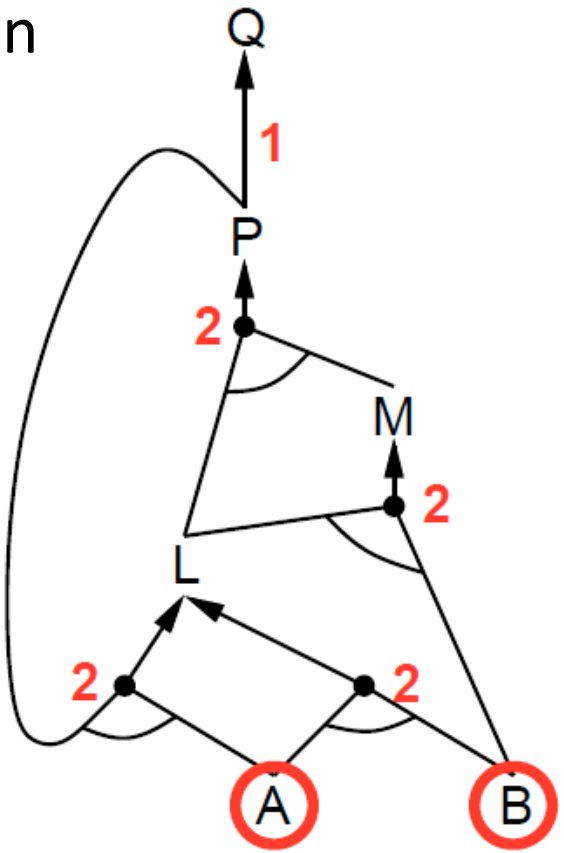
$$A$$

$$B$$

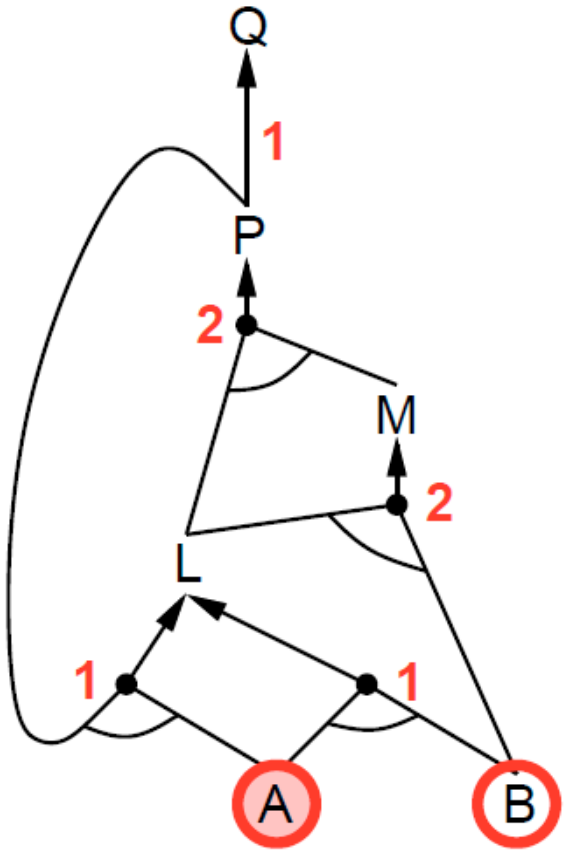


Unidad 2. Reglas en la minería de datos
Ejemplo encadenamiento hacia adelante

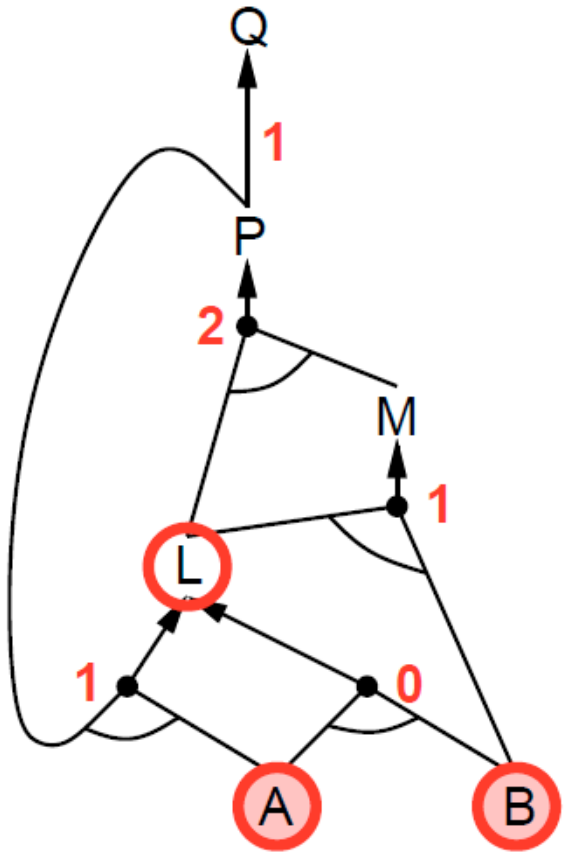
- Los hechos conocidos en la KB son A y B
- Se evalúan las reglas cuyos antecedentes estén contenidos en la KB (A y B)



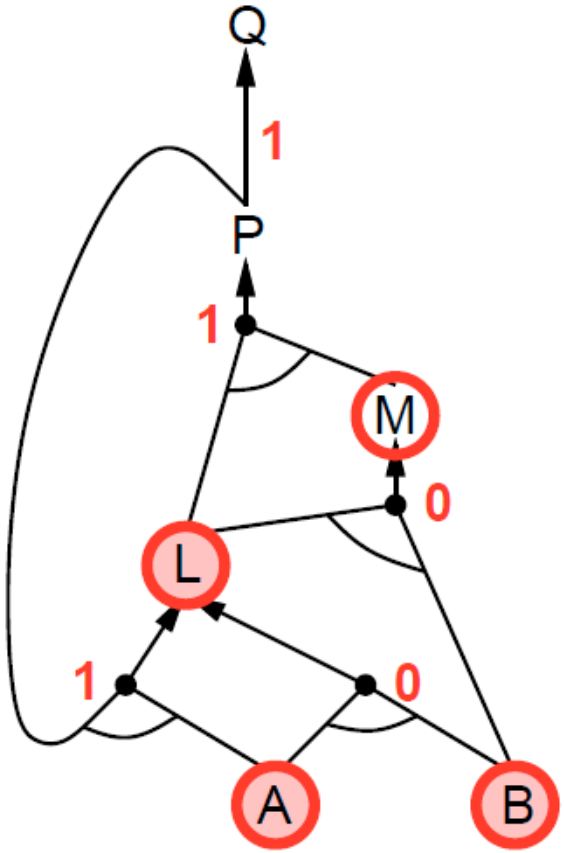
- Se cumple la regla: $A \wedge B \Rightarrow L$



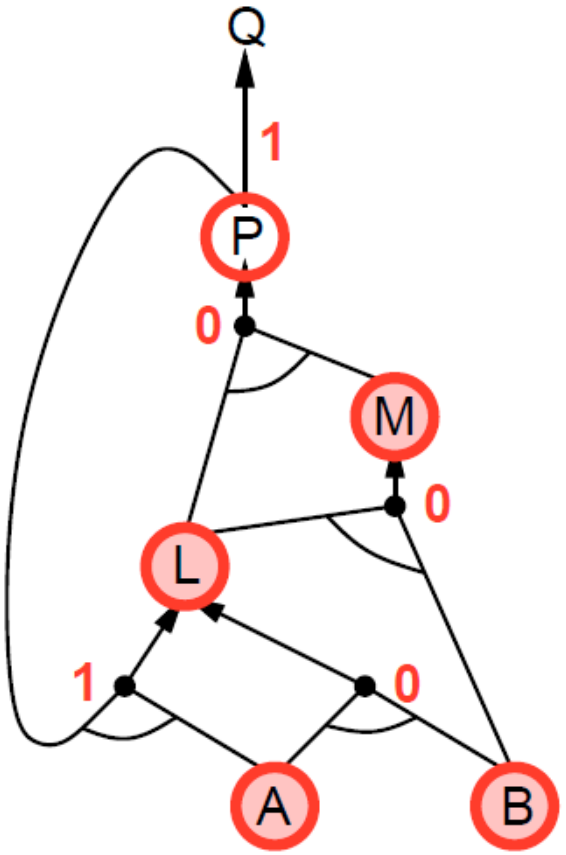
- Se añade L a la KB



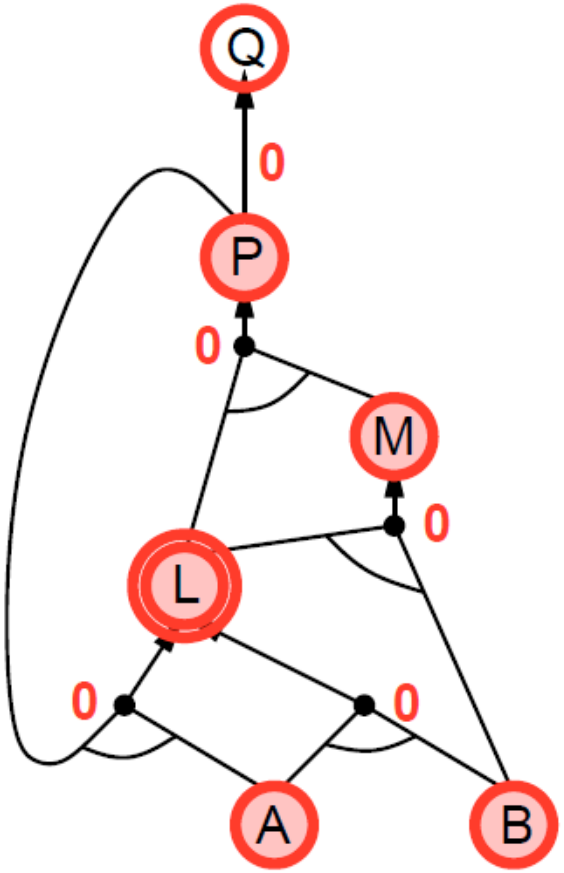
- Se evalúa B ∧ L ⇒ M
- Se añade M a la KB



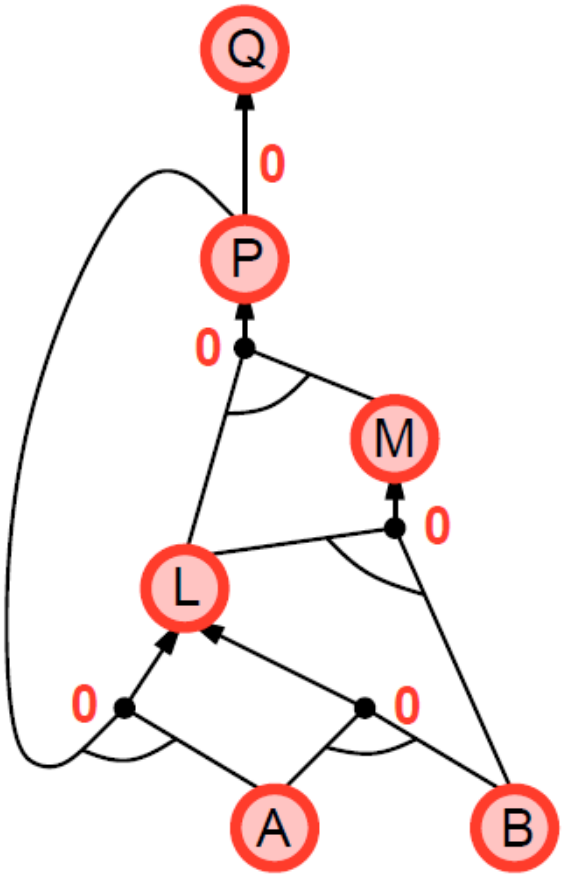
- Se evalúa L ∧ M ⇒ P
- Se añade P a la KB



- Se evalúa P ⇒ Q



- Se añade Q a la KB
- Fin, se ha llegado al objetivo Q



UNIVERSIDAD DE MURCIA

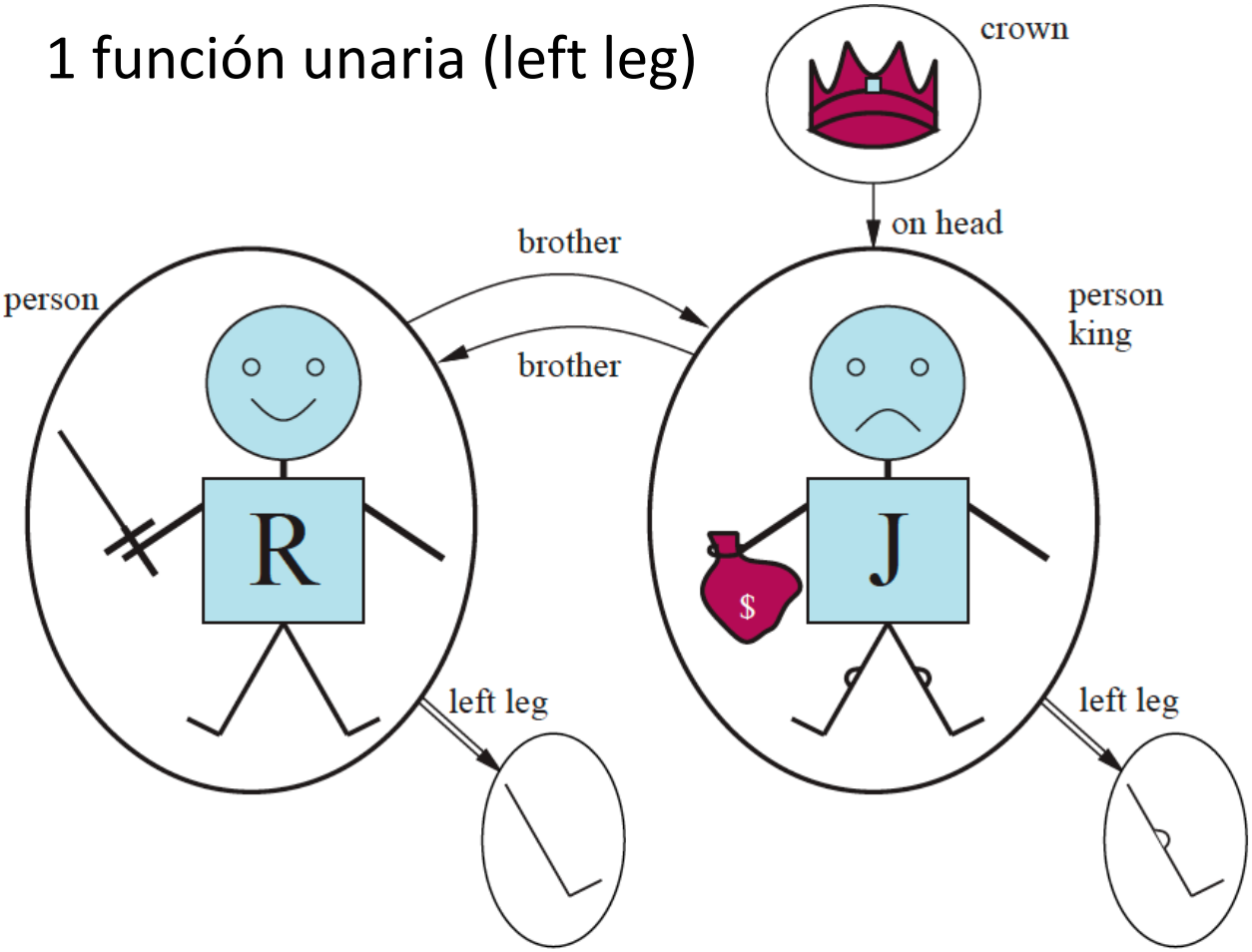
Unidad 2. Reglas en la minería de datos

LÓGICA DE PRIMER ORDEN

- La lógica proposicional es demasiado simple para expresar un mundo real lleno de relaciones
- Lenguajes naturales vs. lenguajes formales
- Lenguajes declarativos vs. lenguajes imperativos
- La LPO busca combinar lo mejor de ambos mundos
  - Objetos: representan objetos de un mundo concreto (personas, casas, números, Ronald…)
  - Predicados: representan relaciones que toman objetos como argumentos y devuelven un valor de verdad (hermano de, más grande que, parte de…)
  - Funciones: mapeos de objetos a otros objetos o valores (el padre de, el mejor amigo de…)
- La principal diferencia es en el compromiso ontológico (la naturaleza de la realidad) del lenguaje

- Para cualquier mundo
  - Variables (x, y, z): cualquier objeto del mundo. No es específico para ningún mundo concreto
  - T,F: verdad y falso
  - Conectores lógicos: los mismos que en la LP
  - Cuantificadores universal (∀) y existencial (∃)
  - Igualdad (=)
- Para un mundo específico
  - Constantes (Juan, A, B): representa un objeto único del mundo
  - Predicado (Padre, Rápido, R, P): representan relaciones:
    - Si es unaria: es una propiedad
    - Por ej: Madre (Juan, María), Estudiante (María), Persona (x)…
  - Función: representa un objeto ”sin nombre”
    - Corona (María) [¡¡No es una subrutina que devuelve algo!!]

- ¿Qué vemos aquí?
  - Número de objetos 5 objetos
  - Relaciones 3 relaciones unarias (person, king y crown)
    2 relaciones binarias (brother y on head)
  - Funciones 1 función unaria (left leg)



Unidad 2. Reglas en la minería de datos
Ejercicio LPO: Dominio de la Familia

- Ejercicio 3. Dominio de la Familia:
  - La madre de uno es su progenitora femenina.

$$\text{LPO: } \forall x, y \ (\text{Madre}(x, y) \leftrightarrow (\text{Progenitor}(x, y) \wedge \text{Mujer}(x)))$$

  - El esposo de uno es su cónyuge masculino.
  - Varón y mujer son categorías disjuntas (excluyentes).
  - Progenitor e hijo son relaciones inversas.
  - Un abuelo es el progenitor del progenitor de uno.
  - Un hermano es otro hijo de los progenitores de uno.

UNIVERSIDAD DE MURCIA

Unidad 2. Reglas en la minería de datos

PROGRAMACIÓN DECLARATIVA



- En la IA se ha utilizado la lógica para representar
- Se han automatizado estos procesos mediante lenguajes de programación declarativos
- El programador indica **qué** hay que hacer pero **no cómo** se hace
- Se usan sentencias similares a las que hemos visto en LPO

- Lenguaje para la programación lógica, basado en LPO
- En mayúsculas las variables (X) y minúsculas las constantes
  - LPO: $Student(x) \wedge InMineria(x) \Rightarrow Clever(x)$
  - PROLOG: clever(X) :- student(X), inMineria(X).

```
mother_child(trude, sally).

father_child(tom, sally).
father_child(tom, erica).
father_child(mike, tom).

sibling(X, Y):-parent_child(Z, X), parent_child(Z, Y).

parent_child(X, Y) :- father_child(X, Y).
parent_child(X, Y) :- mother_child(X, Y).
```

```
?- sibling(sally, erica).
 Yes
```

- Lenguaje para la programación de sistemas expertos mediante reglas
- Es el que usaremos en las prácticas

```
(deftemplate persona
   (slot nombre)
   (slot edad)
 )
```

```
(deffacts personas_empadronadas
    (persona (nombre "Manolo")  (edad 54))
    (persona (nombre "Ignacio") (edad 18))
    (persona (nombre "Marisa")  (edad 34))
 )
```

```
(defrule censar
    (persona (nombre ?n) (edad ?e))
    (test (> ?e 17))
    =>
    (assert (elector (nombre ?n)))
)
```